Технически университет – София, ФКСТ

Дисциплина:	„Компютърни архитектури“		
Специалност:	„КСИ“, III курс (V семестър), гр. 43	Дата:	04.11.2026
Студент:	**Алекс Огнянов**	Фак. №	**121224052**

Отчет по проведено лабораторно упражнение № 3
„Изследване на конвейерни компютърни архитектури (Част 2)“

Цел: Изследване и оптимизация на комуникиращите си процеси в конвейерни компютърни архитектури в условията на конвейерни състезания.

1. Изследване на 5-фазен линеен конвейер

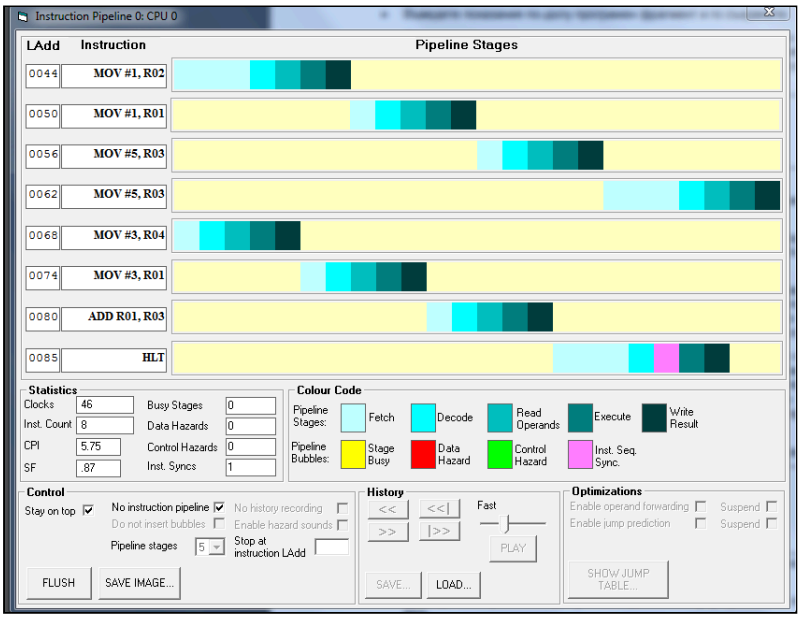
по заданието

```
MOV #1, R01
MOV #5, R03
MOV #3, R01
ADD R01, R03
HLT
```

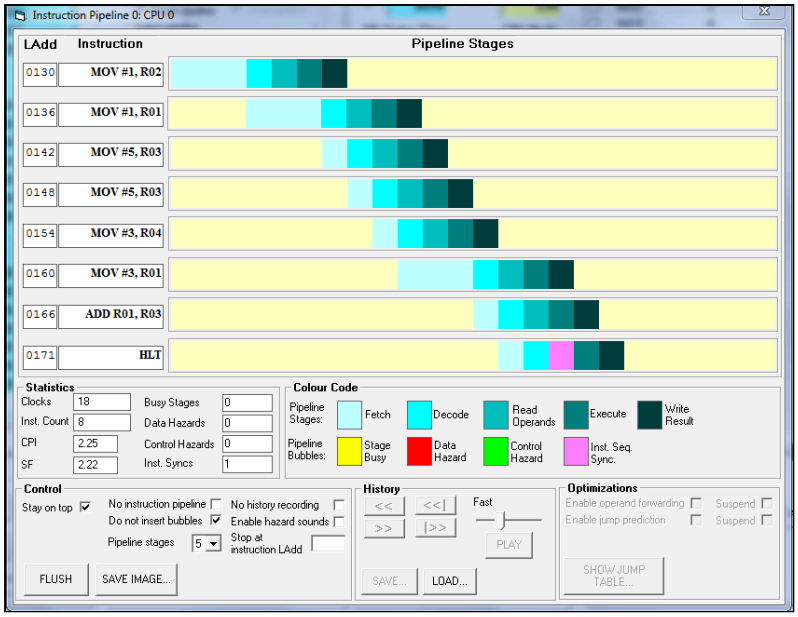
изпълнено в симулатора

```
MOV #1, R02
MOV #1, R01
MOV #5, R03
MOV #5, R03
MOV #3, R04
MOV #3, R01
ADD R01, R03
HLT
```

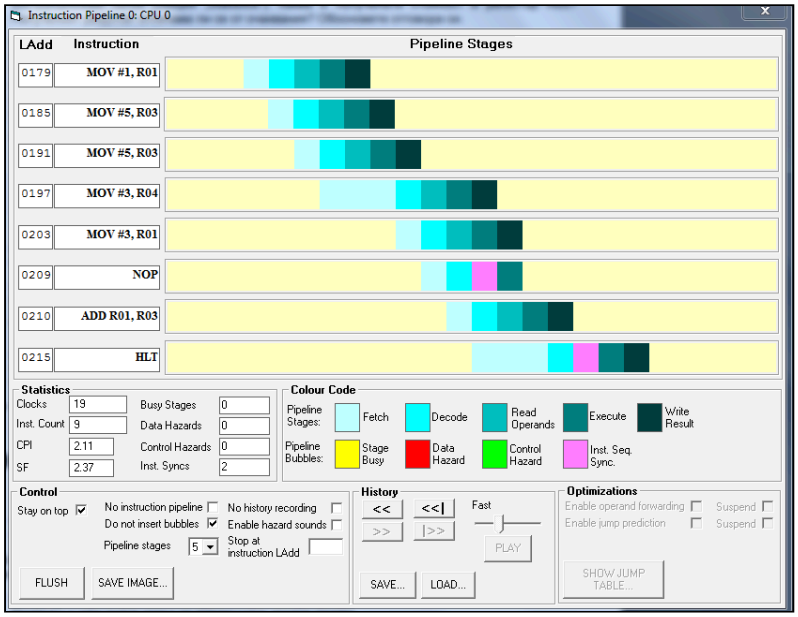
Очакваната стойност на R03 е 8 (5 + 3). Изпълнената в симулатора последователност съдържа три допълнителни инструкции спрямо заданието (MOV #1, R02, повторен MOV #5, R03 и MOV #3, R04), които не променят логиката на изчислението в R03. Програмата е изследвана последователно в четири режима.



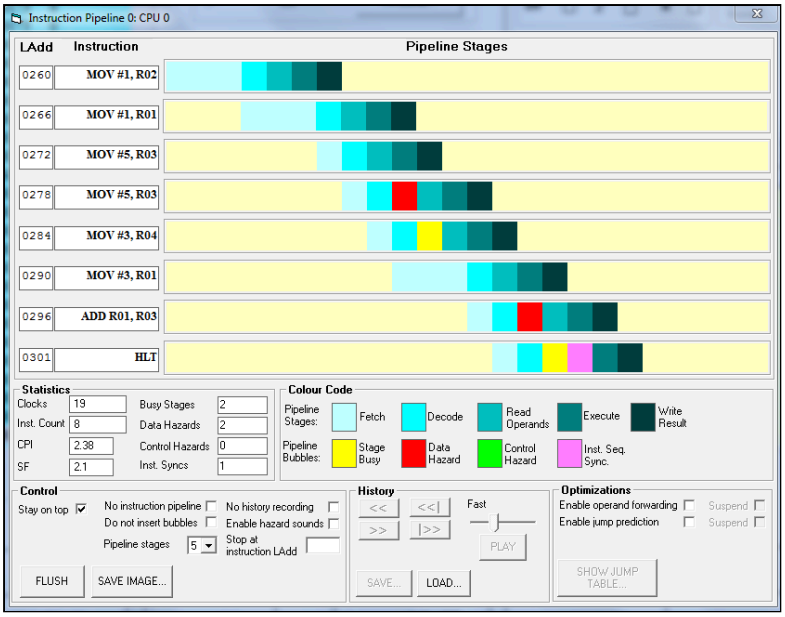
Фиг. 1. „No instruction pipeline“: последователно изпълнение



Фиг. 2. „Do not insert bubbles“: конвейер без изчакване



Фиг. 3. „Do not insert bubbles“ и вмъкната инструкция NOP (първият ред е извън видимата част)



Фиг. 4. Конвейер с автоматично вмъкване на мехури

Показател	Без конвейер	Без мехури	Без мехури + NOP	С мехури
Clocks (тактове)	46	18	19	19
Inst. Count (инструкции)	8	8	9	8
CPI	5,75	2,25	2,11	2,38
SF (Speed-up Factor)	0,87	2,22	2,37	2,1
Busy Stages	0	0	0	2
Data Hazards	0	0	0	2
Control Hazards	0	0	0	0
Inst. Syncs	1	1	2	1
R03 (определено от времедиаграмата)	8	**6**	8	8

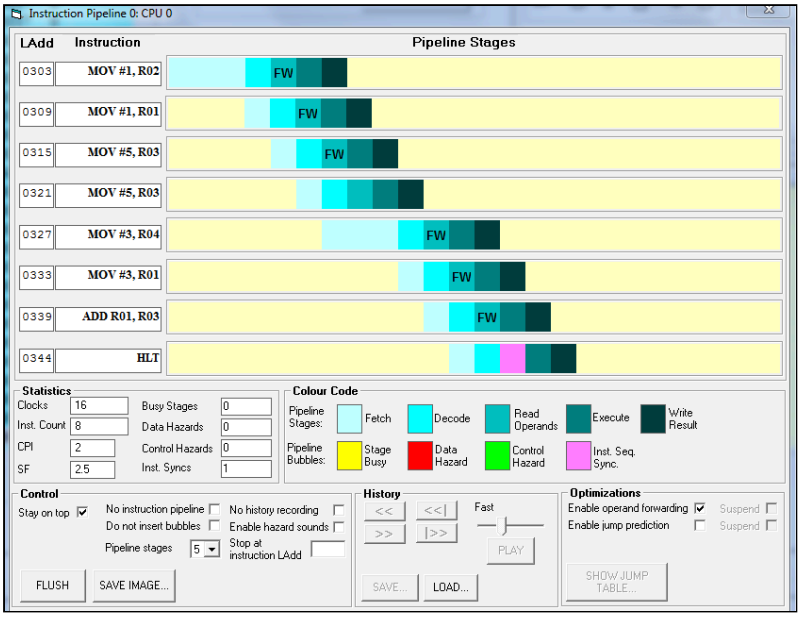
Стойността на R03 не се вижда на снетите екрани и е определена от времедиаграмите.

- **Без конвейер.** Инструкциите се изпълняват строго последователно за 46 такта (CPI = 5,75, SF = 0,87). Резултатът е очакваният, R03 = 8.
- **„Do not insert bubbles“.** ADD R01, R03 чете операндите си в същия такт, в който MOV #3, R01 е във фаза Execute, т.е. преди записа в R01. Прочита се старата стойност 1 и R03 = 5 + 1 = 6. Резултатът е грешен, макар изпълнението да е най-кратко за конвейера без оптимизации (18 такта).
- **С NOP.** Инструкцията NOP отмества четенето на R01 с един такт, така че то съвпада с фазата Write Result на MOV #3, R01. Резултатът отново е R03 = 8, с цена 19 такта и една допълнителна инструкция.
- **С мехури.** Симулаторът сам вмъква мехури: Data Hazards = 2 и Busy Stages = 2. Първият мехур е при втората MOV #5, R03, а вторият задържа ADD R01, R03 до записа на R01.

Резултатът е R03 = 8 за 19 такта. Спрямо режима без конвейер времето намалява 2,42 пъти.

Следователно NOP и автоматичните мехури дават едно и също време (19 такта), но мехурите не изискват промяна на програмата. По-ниската стойност на CPI при NOP (2,11 срещу 2,38) се дължи единствено на по-големия брой инструкции.

2. Operand forwarding



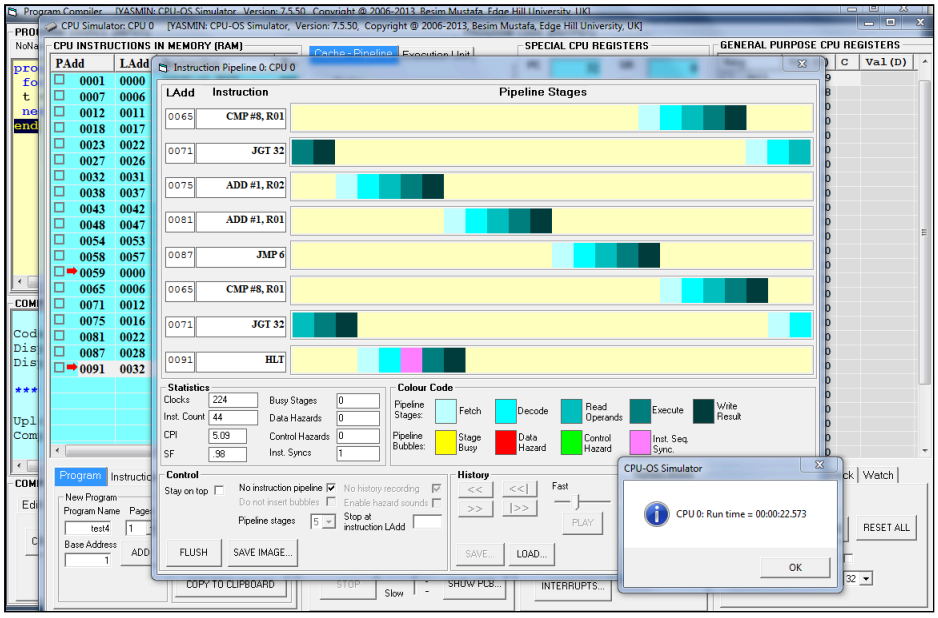
Фиг. 5. Включен operand forwarding (маркировка FW във времедиаграмата)

Показател	С мехури	С operand forwarding
Clocks (тактове)	19	16
Inst. Count (инструкции)	8	8
CPI	2,38	2
SF (Speed-up Factor)	2,1	2,5
Busy Stages	2	0
Data Hazards	2	0
Control Hazards	0	0
Inst. Syncs	1	1

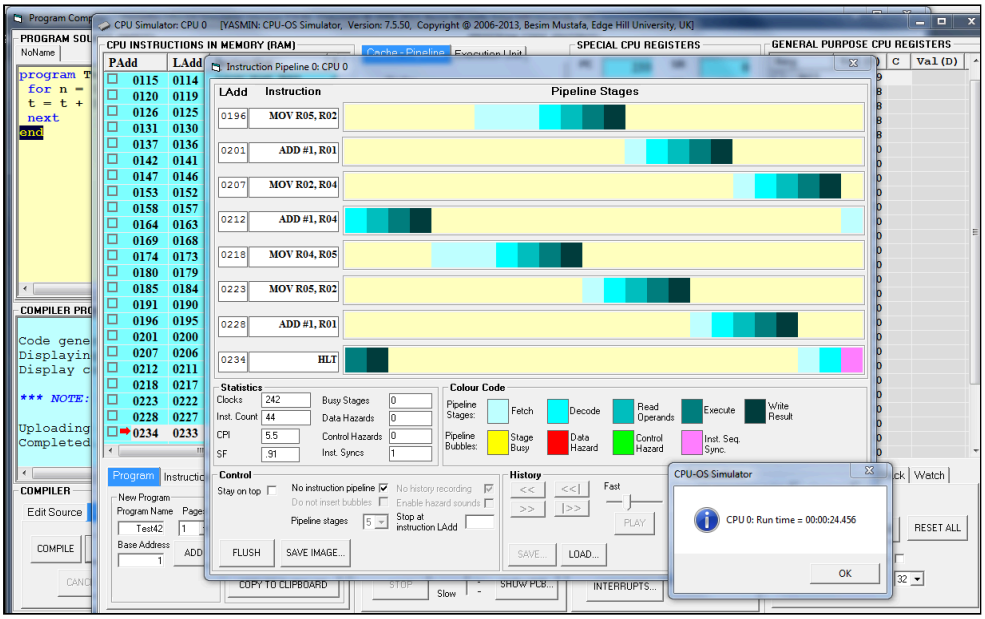
С включен operand forwarding резултатът от изпълнението се подава директно на следващата инструкция, без да се изчаква записът в регистъра. Конфликтите по данни и заетите етапи изчезват (от 2 на 0), R03 остава 8, а времето намалява от 19 на 16 такта, т.е. с 15,8%. CPI спада от 2,38 на 2, а SF нараства от 2,1 на 2,5. Това е единственият изследван режим, който едновременно запазва коректния резултат и скъсява изпълнението.

3. Loop unrolling

```
program Pipeline_Ex4_1
  for n = 1 to 8
    t = t + 1
  next
end
```



Фиг. 6. Pipeline_Ex4_1 (в симулатора test4) с цикъл



Фиг. 7. Pipeline_Ex4_2 (в симулатора Test42) с loop unrolling

Показател	Цикъл (test4)	Loop unrolling (Test42)
Clocks (тактове)	224	242
Inst. Count (инструкции)	44	44
CPI	5,09	5,5
SF (Speed-up Factor)	0,98	0,91
Busy Stages	0	0
Data Hazards	0	0
Control Hazards	0	0
Inst. Syncs	1	1
Размер на кода, байтове	33	234
Време за изпълнение	22,573 s	24,456 s

Генерираният код е 33 байта при цикъла и 234 байта при loop unrolling, т.е. 7,1 пъти по-голям. И при двете програми се изпълняват по 44 инструкции. Цикълът се превежда до CMP, JGT, две инструкции ADD и JMP на итерация, а развитият вариант повтаря тялото без преходи, но с повече инструкции MOV за t = t + 1.

И двата екрана са снети в режим „No instruction pipeline“, затова Control Hazards = 0 и ефектът от премахването на преходите не може да се наблюдава. В този режим Test42 е с 8% по-бавен (242 срещу 224 такта). Разликата съответства на 11 срещу 2 удължени фази Fetch, тъй като всички останали инструкции отнемат по 5 такта. В конвейерен режим очакването е премахването на JGT и JMP да намали конфликтите по управление, срещу което се заплаща с по-голям код.

4. Re-arranging instruction sequence

изходен код

```
program Pipleine_Ex5_1
  a = 1
  b = a
  c = 2
end
```

очакван ред (изведен от кода)

```
MOV #1, R03
MOV R03, R01
MOV R01, R04
MOV R04, R02
MOV #2, R05
MOV R05, R03
HLT
```

снета последователност

```
MOV #1, R03
MOV R03, R01
MOV R01, R04
MOV #2, R05
MOV R04, R02
MOV R05, R03
HLT
```



Фиг. 8. Пренаредена инструкционна последователност, режим „No instruction pipeline“

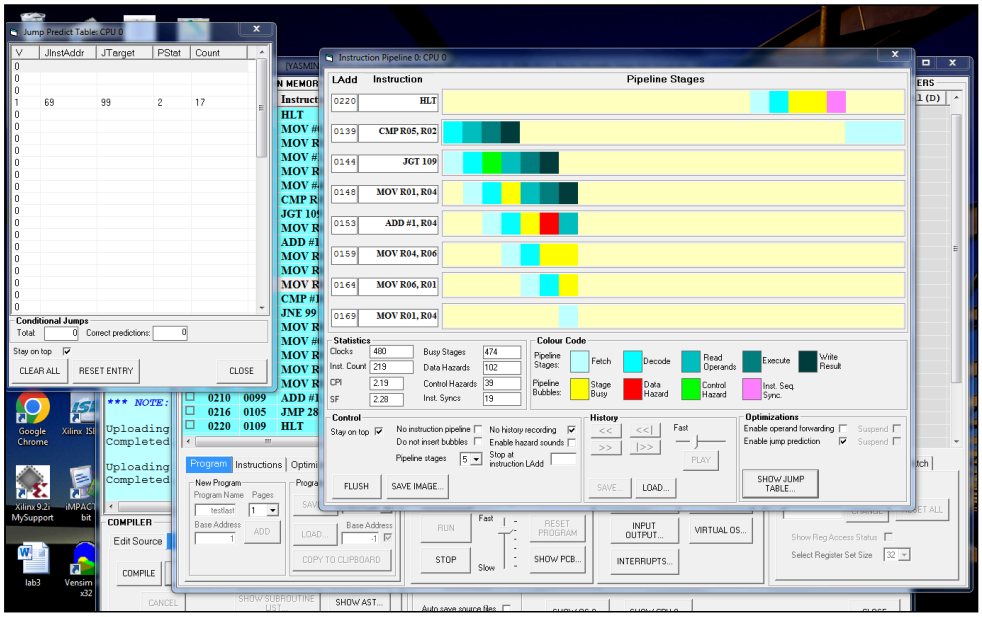
Показател	Пренаредена последователност (zadabc)
Clocks (тактове)	39
Inst. Count (инструкции)	7
CPI	5,57
SF (Speed-up Factor)	0,9
Busy Stages	0
Data Hazards	0
Control Hazards	0
Inst. Syncs	1

В снетата последователност независимата инструкция MOV #2, R05 е преместена между зависимите MOV R01, R04 и MOV R04, R02. Логиката на програмата не се променя, защото MOV #2, R05 не чете и не записва R01, R04 и R02, а всички променливи получават същите стойности. Целта на пренареждането е да раздели зависимите инструкции, така че между записа и четенето на R04 да има допълнителен такт и да се вмъкват по-малко мехури.

Липсващи данни. Наличен е само екранът на пренаредената последователност (съответстваща на Pipleine_Ex5_2) в режим „No instruction pipeline“. Стойностите на CPI, SF, Data Hazards и Control Hazards за Pipleine_Ex5_1, Pipleine_Ex5_3 и Pipleine_Ex5_4 не са снети, затова подобрението в проценти не е изчислено.

5. Jump prediction

```
program Pipeline_Ex6
  i = 0
  for p = 1 to 40
    i = i + 1
    if i = 10 then
      i = 0
      r = i
    end if
  next
end
```



Фиг. 9. Pipeline_Ex6 (в симулатора testlast) с jump prediction и таблицата Jump Predict Table

Показател	С jump prediction (междинно състояние)
Clocks (тактове)	480
Inst. Count (инструкции)	219
CPI	2,19
SF (Speed-up Factor)	2,28
Busy Stages	474
Data Hazards	102
Control Hazards	39
Inst. Syncs	19

V	JInstAddr	JTarget	PStat	Count
1	69	99	2	17

Екранът е снет при активен бутон STOP, т.е. по време на изпълнението, затова статистиката отразява междинно състояние (219 инструкции и 480 такта). Записите в Jump Predict Table са: V (валиден запис), JInstAddr (адрес на инструкцията за преход), JTarget (адрес на целта), PStat (състояние на предсказването) и Count (брой обръщения към записа). Единственият запис е за JNE 99 на логически адрес 69, който прескача тялото на условието „if i = 10“ и се извършва при 9 от 10 итерации. В момента на снемане броячите Total и Correct predictions показват 0.

Липсващи данни. Няма екран на изпълнението без jump prediction и краен резултат с включено предсказване, затова двете изпълнения не са съпоставени количествено.

Обобщени изводи

- Конвейерът ускорява изпълнението 2,42 пъти спрямо последователното, но без защита от зависимостите по данни резултатът е грешен (R03 = 6 вместо 8).
- Мехурите и NOP запазват коректността с цена 19 такта, а operand forwarding я запазва и намалява времето на 16 такта (CPI = 2).
- Loop unrolling увеличава кода 7,1 пъти. Ползата му за конфликтите по управление не може да бъде измерена при изключен конвейер.
- Пренареждането на инструкции не променя логиката на програмата. Ефектът му върху производителността и този на jump prediction не са оценени количествено поради липсващи измервания.